PDF Examination

Scenario Overview

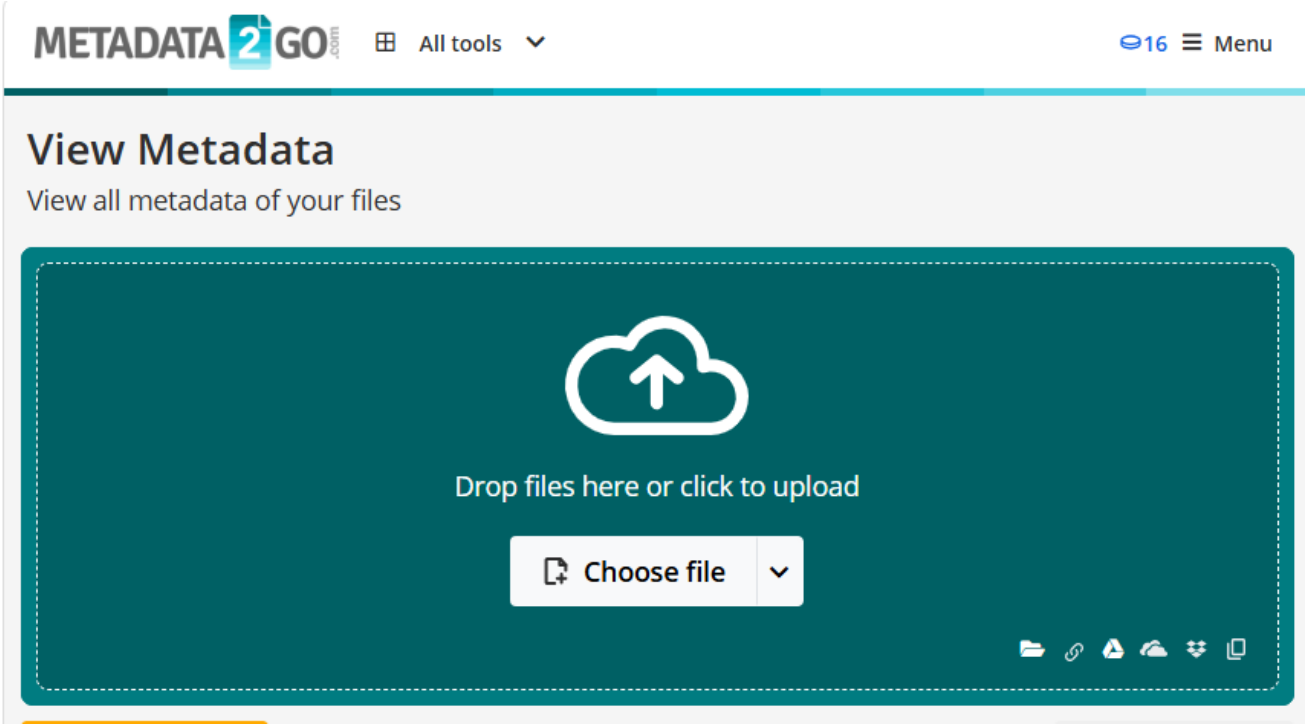
Our analysts managed to get hold of a document that we believe has a flag on it. Can you recover it?

Tasks

1. **Analyze the PDF metadata**
    - Upload the file to a metadata viewer such as Metadata2Go.
    - Identify the **creator_tool** field to determine which program exported the PDF.
    - Identify the **pdf_version** field to determine the PDF specification used.
2. **Identify the redaction and watermarking software**
    - Open the PDF in a PDF editor.
    - Inspect the black redaction boxes and watermark.
    - Determine which software was used to apply the redaction and watermark.
3. **Recover hidden content**
    - Attempt to remove or move the black redaction boxes.
    - Alternatively, highlight the area behind the boxes to copy any underlying text that was not properly removed.
    - Use a PDF parser to inspect document objects if needed.
4. **Extract the flag**
    - Locate the hidden text revealed through redaction removal or parsing.
    - Document the recovered flag.

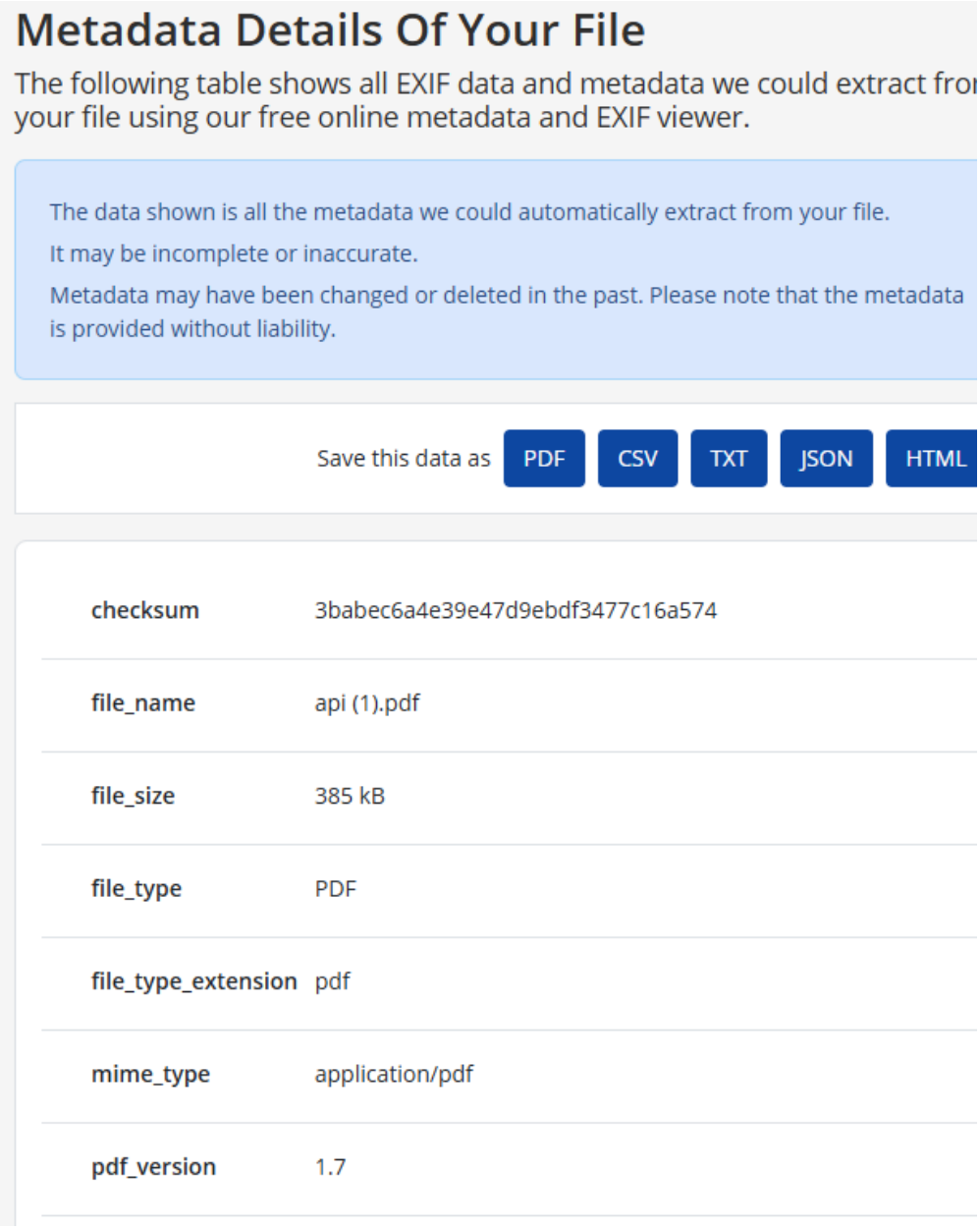
1. What is the name of the program that exported this PDF file?

Navigate to the site METADATA2GO https://www.metadata2go.com/view-metadata and load the file provided in the challenge by clicking “Choose File.”



A long list of data will appear. Here under “Creator” we see the name of the program that exported the PDF file.

Answer: Adobe



Metadata Details Of Your File

The following table shows all EXIF data and metadata we could extract fro
your file using our free online metadata and EXIF viewer.

The data shown is all the metadata we could automatically extract from your file.

It may be incomplete or inaccurate.

Metadata may have been changed or deleted in the past. Please note that the metadata is provided without liability.

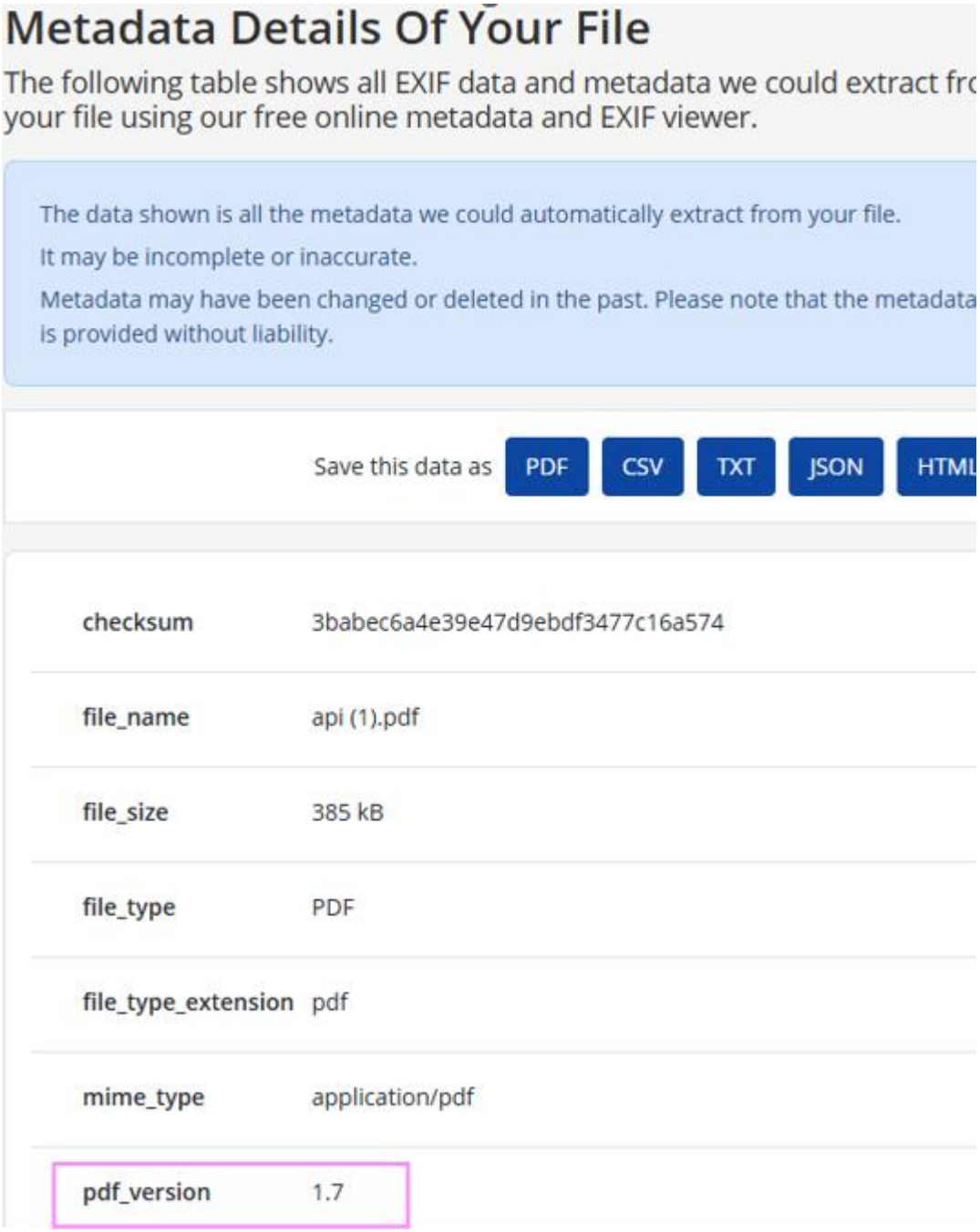
Save this data as PDF CSV TXT JSON HTML

checksum	3babec6a4e39e47d9ebdf3477c16a574
file_name	api (1).pdf
file_size	385 kB
file_type	PDF
file_type_extension	pdf
mime_type	application/pdf
pdf_version	1.7

2. What PDF version is this file?

In the same list we can find the pdf version.

Answer: 1.7



Metadata Details Of Your File

The following table shows all EXIF data and metadata we could extract fro
your file using our free online metadata and EXIF viewer.

The data shown is all the metadata we could automatically extract from your file.
It may be incomplete or inaccurate.
Metadata may have been changed or deleted in the past. Please note that the metadata
is provided without liability.

Save this data as PDF CSV TXT JSON HTML

checksum	3babec6a4e39e47d9ebdf3477c16a574
file_name	api (1).pdf
file_size	385 kB
file_type	PDF
file_type_extension	pdf
mime_type	application/pdf
pdf_version	1.7

3. What software was used to redact the file and insert a watermark?

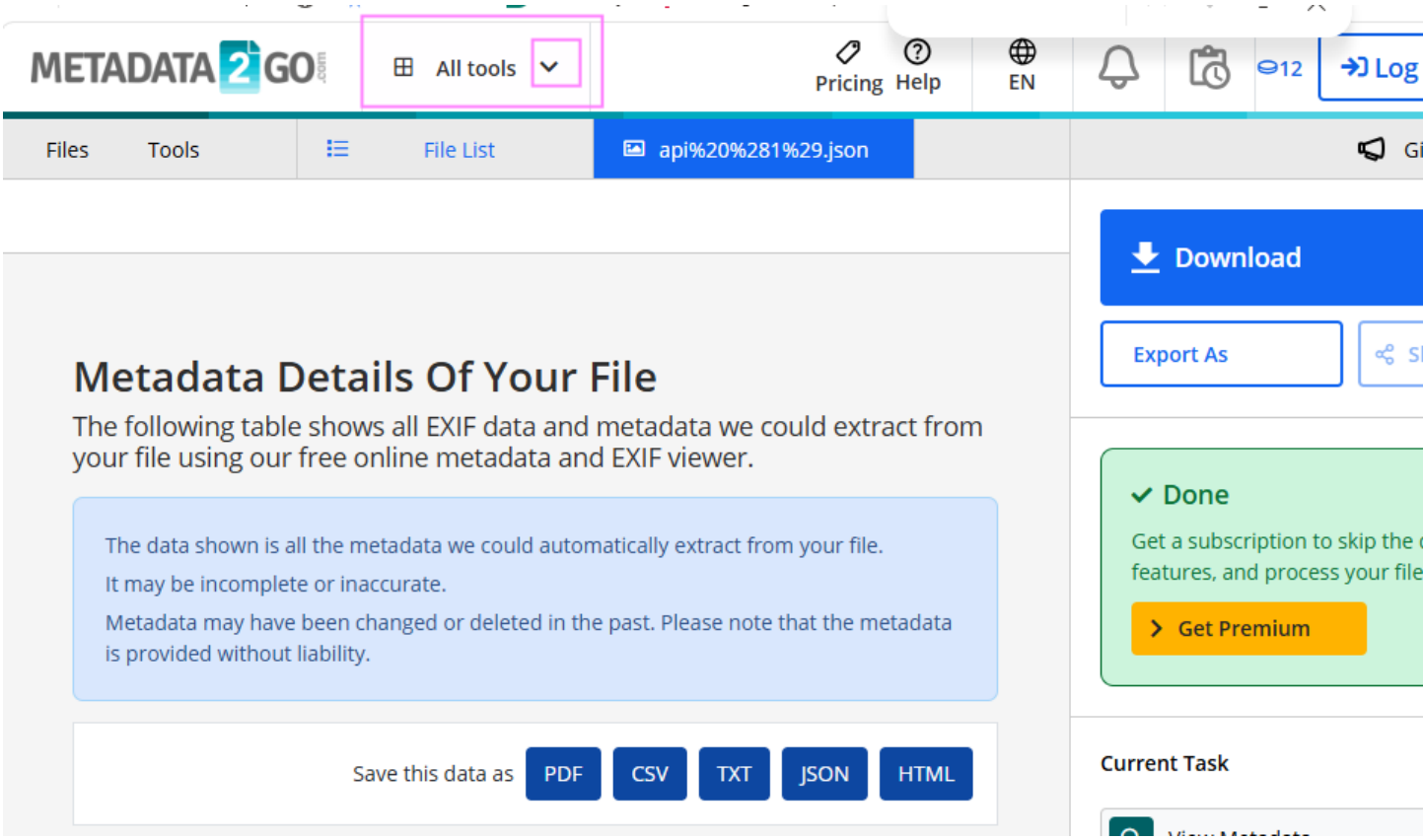
Scroll down to “producer.”

Answer: PDFTron

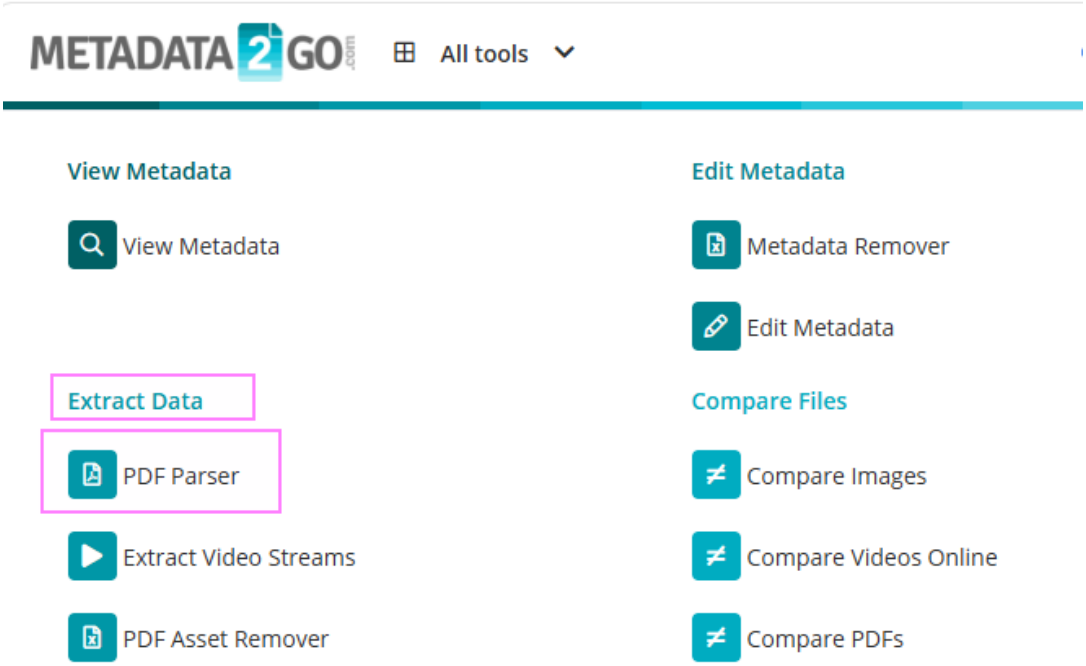
producer PDFTron built-in office converter, V7.0.5.73931

4. What is the flag?

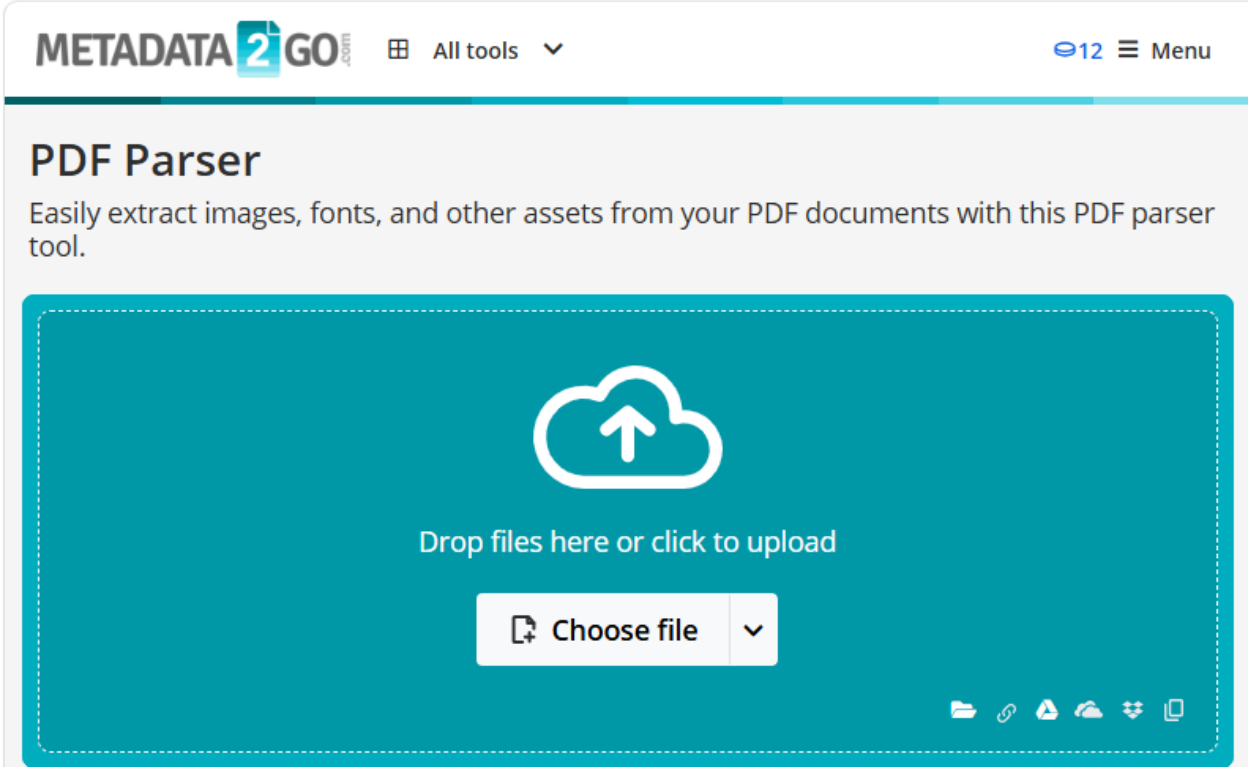
Click the drop-down menu “All tools.”



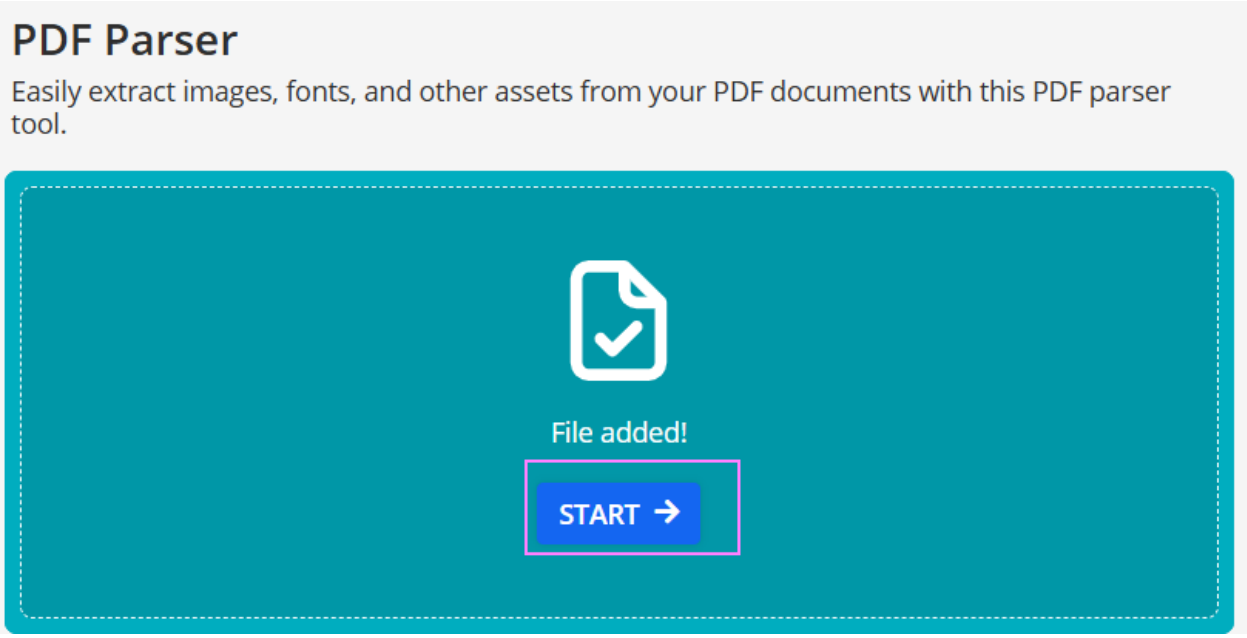
Under “Extract Data” select “PDF Parser.”



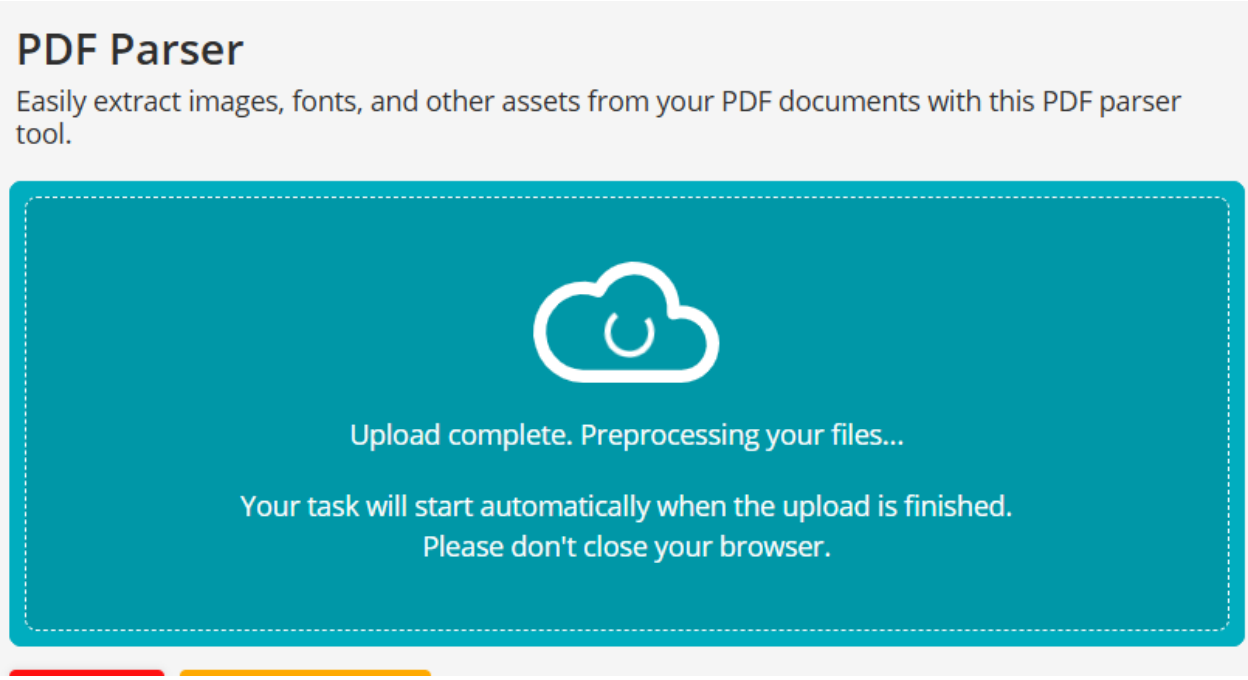
Load the same file again.



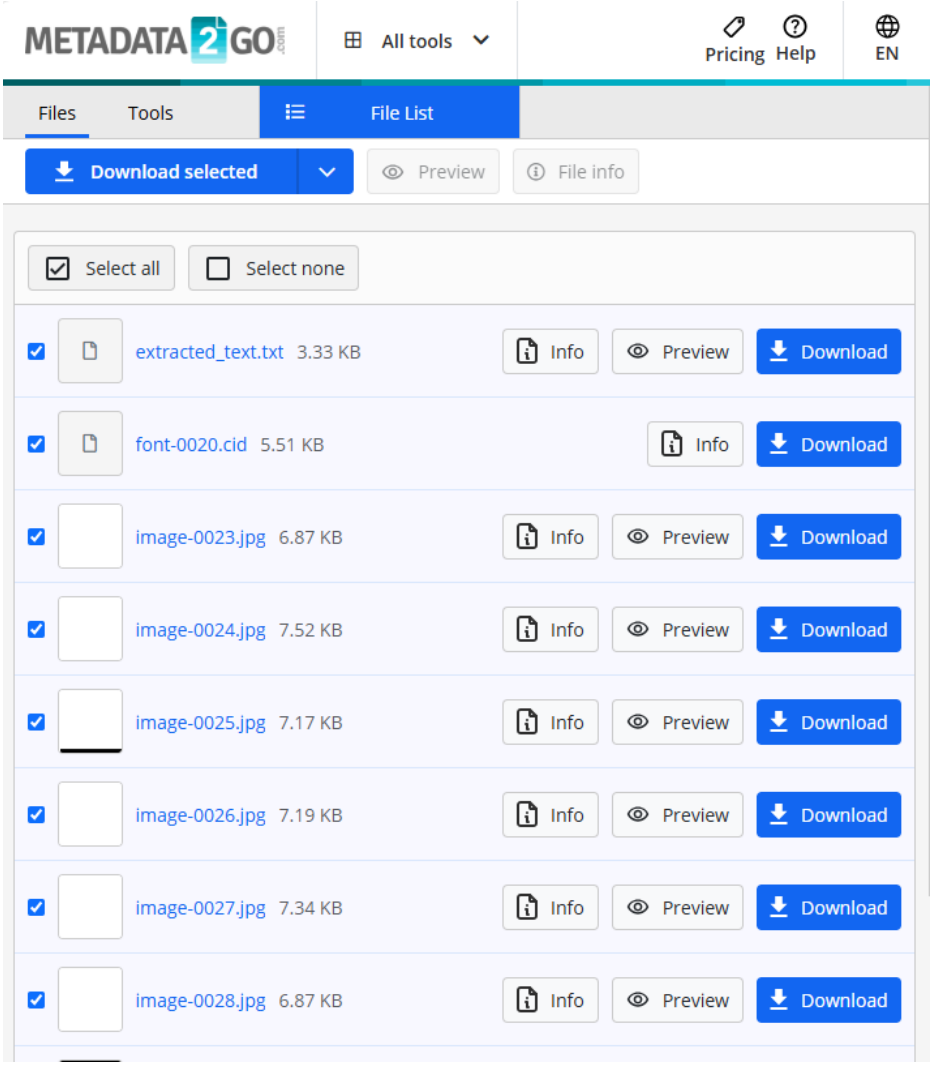
Select “Start.”



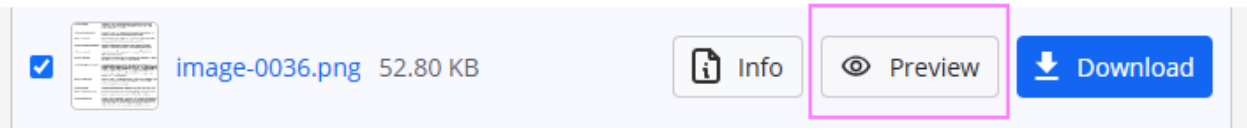
You will then see the file processed.



Once the file is processed you will then see multiple files.



Scroll down to the last document and click “Preview.”



Search for the flag.

Answer: SKY-PDRD-2390

Liber8tion TLS API

Methods

tlsSocket.address()	Returns the bound address, the address family name, and port of the underlying socket as reported by the operating system: { port: 12346, family: 'IPv4', address: '127.0.0.1' }.
tlsSocket.authorizationError	Returns the reason why the peer's certificate was not been verified. This property is set only when tlsSocket.authorized === false.
tlsSocket.authorized	Returns true if the peer certificate was signed by one of the CAs specified when creating the tls.TLSSocket instance, otherwise false.
tlsSocket.disableRenegotiation()	Disables TLS renegotiation for this TLSSocket instance. Once called, attempts to renegotiate will trigger an 'error' event on the TLSSocket.
tlsSocket.encrypted	Always returns true. This may be used to distinguish TLS sockets from regular net.Socket instances.
tlsSocket.getCipher()	Returns an object representing the cipher name. The version key is a legacy field which always contains the value 'TLSv1/SSLv3'.
tlsSocket.getEphemeralKeyInfo()	Returns an object representing the type, name, and size of parameter of an ephemeral key exchange in Perfect Forward Secrecy on a client connection. It returns an empty object when the key exchange is not ephemeral. As this is only supported on a client socket; null is returned if called on a server socket. The supported types are 'DH' and 'ECDH'. The name property is available only when type is 'ECDH'.
tlsSocket.getFinished()	Returns: <Buffer> | <undefined> The latest Finished message that has been sent to the socket as part of a SSL/TLS handshake, or undefined if no Finished message has been sent yet.
tlsSocket.getFlag()	Returns an string representing the flag. Default value is `SKY-PDRD-2390`
tlsSocket.getPeerCertificate()	Returns an object representing the peer's certificate. The returned object has some properties corresponding to the fields of the certificate.
tlsSocket.getProtocol()	Returns a string containing the negotiated SSL/TLS protocol version of the current connection. The value 'unknown' will be returned for connected sockets that have not completed the handshaking process. The value null will be returned for server sockets or disconnected client sockets.
tlsSocket.getSession()	Returns the TLS session data or undefined if no session was negotiated. On the client, the data can be provided to the session option of tls.connect() to resume the connection. On the server, it may be useful for debugging.
tlsSocket.getTLSTicket()	For a client, returns the TLS session ticket if one is available, or undefined. For a server, always returns undefined.
tlsSocket.localAddress	Returns the string representation of the local IP address.